

SIG-RECETTE

Payment Structure, Procedure & Workflow

Laravel 10 · Livewire 3 · Symfony Workflow

Cash · Cheque · Mobile Money (QOSIC / FedaPay / PayGate)

SIG-RECETTE — Payment Structure, Procedure & Workflow

Audience: Developers, integrators, accounting agents and project administrators of SIG-RECETTE. **Scope:** Complete description of the payment domain — data model, state machine, services, mobile-money integration, synchronization, notifications and end-to-end procedure. **Stack:** Laravel 10 / PHP 8.2 · Livewire 3 · Symfony Workflow (zerodahero/laravel-workflow) · Spatie Permissions (French strings) · Barryvdh DomPDF.

Table of contents

1. Overview
2. Domain glossary
3. High-level architecture
4. Data model
5. Enumerations
6. Invoice state machine (workflow)
7. Payment lifecycle
8. Cash & cheque payment procedure
9. Mobile-money / digital payment procedure
10. Per-tax-code balance tracking
11. Receipts and PDF generation
12. Synchronization with the remote collection app
13. Events, listeners, jobs and notifications
14. Authorization and permissions
15. Routes reference
16. Configuration reference
17. Database migrations reference
18. Step-by-step procedures
19. Troubleshooting

1. Overview

SIG-RECETTE manages the **issuance of tax notices (Invoice)** and the **collection of payments (Payment)** for a public-revenue authority. It supports three payment channels:

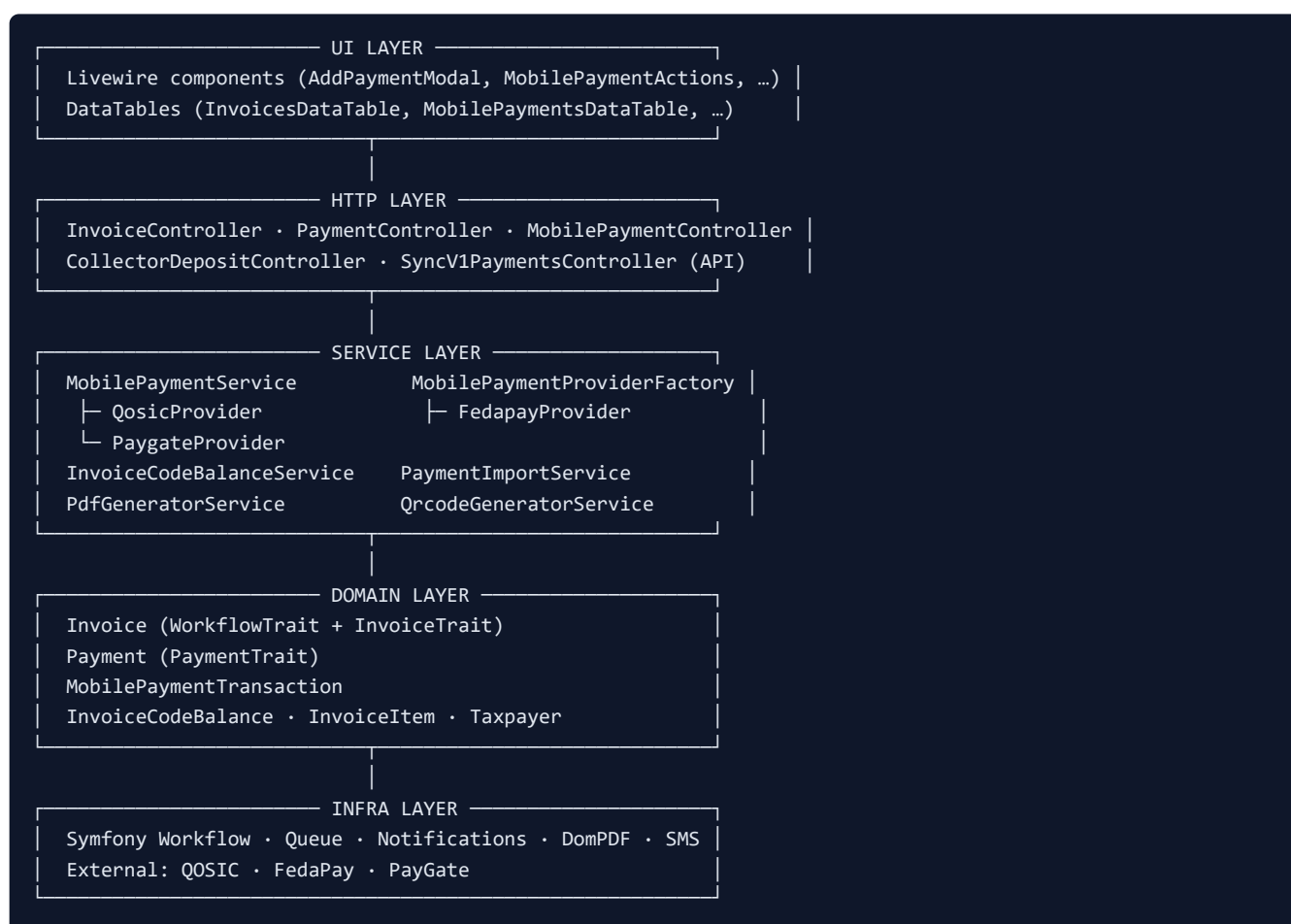
Channel	Code	Description
Cash	CASH	Counter cash payments recorded by an agent
Cheque	CHEQUE	Bank cheque payment with reference
Digital / Mobile money	DIGI	Payment through QOSIC, Fedapay or PayGate (TMONEY / FLOOZ)

Payments are always **attached to an Invoice** and therefore to a **Taxpayer**. The invoice itself moves through a controlled **Symfony Workflow state machine** before it becomes payable, and each payment is **split per tax code** so reporting can be done per revenue line.

2. Domain glossary

Term (FR / EN)	Meaning
Avis / Invoice	Tax notice issued to a taxpayer (Invoice)
Avis sur titre — TITRE	Formal title-based invoice that must be approved before collection
Avis au comptant — COMPTANT	Immediate, on-the-spot invoice (skips the order-number requirement)
Redevable / Taxpayer	The natural or legal person liable for the tax (Taxpayer)
Article taxable / TaxpayerTaxable	A taxable line item attached to a taxpayer (TaxpayerTaxable)
Code	Internal accounting code identifying a tax / revenue line (TaxLabel1.code)
Régisseur	Agent authorised to validate the chain of collection
Receveur / Collector	Agent that physically collects the money
Versement / Deposit	Money handed over by a collector to the accountant
Annulation / Réduction	Special "negative" Payment rows used to cancel or reduce an approved invoice

3. High-level architecture



4. Data model

4.1 Invoice — app/Models/Invoice.php

Main fields:

Field	Type	Purpose
invoice_no	string (unique)	Public invoice number
order_no	string	Order number required for TITRE invoices to become PENDING
taxpayer_id	FK	Taxpayer this notice belongs to
amount	float	Total billed amount
reduce_amount	float	Amount after reduction (when reduced)
qty	float	Quantity (taxable items)
from_date / to_date	date	Coverage period
pay_status	string	OWING · PART PAID · PAID
status	string	Workflow state (see §6)
type	string	TITRE or COMPTANT
delivery	string	NOT DELIVERED / DELIVERED
delivery_date , delivery_to	date / string	Delivery info
edition_state , printed_at	string / datetime	Print tracking
notes	json	previous_invoice_id , remaining_amount , free_text
uuid	string	Sync idempotency key

Key relationships: taxpayer(), payments(), invoiceitems(), taxpayer_taxables(), printFiles().

Key methods (via InvoiceTrait + model):

- can(\$state) / submitToState(\$state) — workflow guards & transitions.
- get_remains_to_be_paid() — remaining amount.
- isValid() — not rejected/canceled/reduced and not already PAID.
- canGetPayment() — invoice is in a state where a payment is allowed.
- sumAmountsByTaxCode(\$invoice) — totals grouped by code.
- getCode(\$id, \$amount, \$paymentData) — splits a payment amount across the invoice tax codes (used to create one Payment row per code).
- returnPaidAndSumByCode(\$invoice) — paid vs billed per code.
- retrieveByUUIDs(array \$uuids) — eager load for export / PDF.

Useful scopes: ofStatus, approved, unpaid, forTaxpayer, inDateRange.

4.2 Payment — app/Models/Payment.php

Main fields:

Field	Type	Purpose
amount	float	Amount paid
payment_type	enum-like string	CASH · CHEQUE · DIGI
invoice_type	string	Denormalised copy of Invoice.type
reference	string	Payment reference (cheque no, transaction ref, ...)
description	string	e.g. Annulation, Réduction, free text
remaining_amount	string	Remaining on the invoice at the moment of the payment
status	string	PENDING · ACCOUNTED · DONE · CANCELED · VERIFYING · FAILED · EXPIRED
code	string	Tax code this payment line is allocated to
taxpayer_id, invoice_id	FK	Targets
user_id	FK	Issuing user
r_user_id	FK	Receiving user (collector)
deposit	float / ref	Collector deposit reference
notes	text	Free notes
uuid	string	Sync idempotency key
provider	string	qosic · fedapay · paygate (DIGI only)
network	string	TMONEY · FL00Z (DIGI only)
phone_number, external_id	string	DIGI only
verification_attempts, last_checked_at, expires_at	mixed	DIGI verification meta

Relationships: invoice(), taxpayer(), user(), r_user(), tax_label(), mobilePaymentTransaction(), stock_transfers() .

Key trait methods (PaymentTrait):

- getPaymentsByStatus(\$invoice_id, \$status)
- getPaid(\$invoice_id) — sum of effective payments
- getPaidNotAccounted(\$invoice_id) — pending deposits
- getRestToPaid(Invoice \$invoice)
- getPrintData() — payments for PDF (excludes PENDING, Annulation, Réduction)

One business payment may produce **several Payment rows** — one per tax code — through Invoice::getCode() .

4.3 MobilePaymentTransaction — app/Models/MobilePaymentTransaction.php

Tracks the conversation with a mobile-money provider for a single DIGI attempt. One transaction can produce several final Payment rows once verified.

Field	Purpose
reference (unique)	Internal reference sent to the provider
invoice_id, taxpayer_id, user_id	Targets
amount (decimal 15,2)	Requested amount
phone_number	Payer phone
provider	qosic · fedapay · paygate
network	TMONEY · FL00Z
external_id	Provider's transaction id
status	pending → verifying → success / failed / expired
verification_attempts, last_checked_at, verified_at, expires_at	Polling metadata
provider_response (json)	Raw last response
meta (json)	E.g. tax code, free metadata

Helpers: `isPending()`, `isVerifying()`, `isSuccess()`, `isFailed()`, `isExpired()`.

4.4 InvoiceItem — app/Models/InvoiceItem.php

`invoice_id`, `taxpayer_taxable_id`, `qty`, `amount`, `ii_tariff`, `ii_seize`. The line items aggregated to compute the invoice total.

4.5 InvoiceCodeBalance — app/Models/InvoiceCodeBalance.php

Materialised view of "how much is owed / paid per tax code" for an invoice. Maintained by `InvoiceCodeBalanceService`. Unique on (`invoice_id`, `code`).

Field	Purpose
invoice_id, taxpayer_id, code, year	Identity
amount_billed	Sum of invoice items for this code
amount_paid	Sum of effective payments for this code (excludes <code>Annulation</code> / <code>Réduction</code>)
remaining_amount	Computed difference
status	OWING · PAID
last_payment_at	Last applicable payment timestamp

4.6 Taxpayer — app/Models/Taxpayer.php

Holds `tnif`, `nif`, `name`, `mobilephone`, `telephone`, `email`, `address`. Drives identification on invoices, SMS notifications and digital payments.

5. Enumerations

All enums live in `app/Enums/` as native PHP 8.1 string enums and are consumed via `->value` (no Eloquent casts — see project notes).

5.1 PaymentTypeEnums

```
CASH    = 'CASH'
CHEQUE  = 'CHEQUE'
DIGI    = 'DIGI'
```

5.2 PaymentStatusEnums

```
PENDING    payment recorded, not yet accounted (also default for DIGI before verification)
VERIFYING  DIGI: provider has accepted, polling in progress
ACCOUNTED  payment has been booked into a deposit / ledger
DONE       fully reconciled
CANCELED   payment voided
FAILED     DIGI: provider rejected / verification failed
EXPIRED    DIGI: transaction expired without confirmation
```

5.3 InvoicePayStatusEnums

```
OWING      = 'OWING'           // nothing paid
PART_PAID  = 'PART PAID'       // some paid
PAID       = 'PAID'           // fully paid
```

5.4 InvoiceStatusEnums (workflow places)

```
DRAFT
P_ACCEPTED      = 'ACCEPTED WITHOUT ORDER NO'
ACCEPTED        = 'ACCEPTED'
REJECTED_BY_OR
PENDING
REJECTED
APPROVED
APPROVED_CANCELLATION = 'APPROVED-CANCELLATION-OR-REDUCTION'
CANCELED        = 'APPROVED-CANCELED'
REDUCED         = 'APPROVED-REDUCED'
```

5.5 Constants — app/Helpers/Constants.php

```
PROVIDER_QOSIC    = 'qosic'
PROVIDER_FEDAPAY  = 'fedapay'
PROVIDER_PAYGATE  = 'paygate'
MOBILE_PROVIDERS  = [qosic, fedapay, paygate]

NETWORK_TMONEY    = 'TMONEY'
NETWORK_FLOOZ     = 'FLOOZ'
MOBILE_NETWORKS   = [TMONEY, FLOOZ]

ANNULATION = 'Annulation' // marker for cancellation Payment rows
REDUCTION  = 'Réduction'  // marker for reduction Payment rows

INVOICE_TYPE_TITRE    = 'TITRE'
INVOICE_TYPE_COMPTANT = 'COMPTANT'
```

6. Invoice state machine (workflow)

The invoice lifecycle is driven by a Symfony state-machine declared in [config/workflow.php](#) and registered via [config/workflow_registry.php](#). The workflow name is `invoice`.

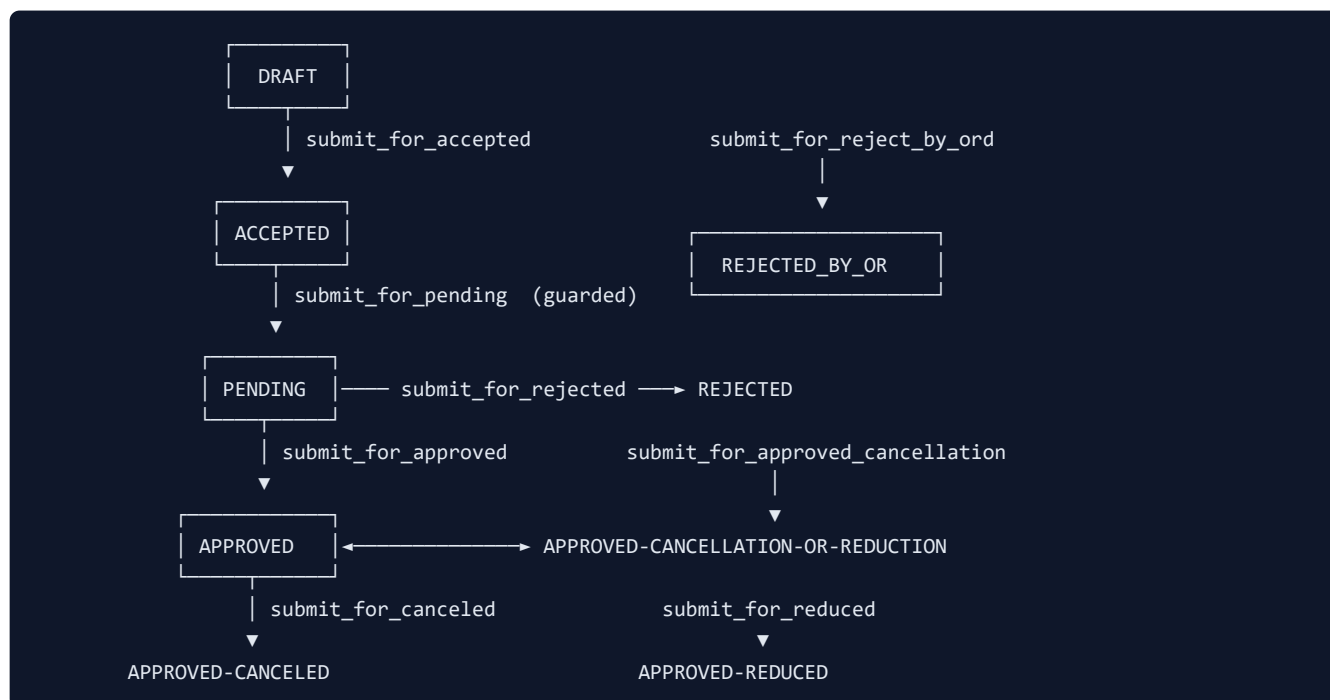
6.1 Places

```
DRAFT
ACCEPTED WITHOUT ORDER NO      (intermediate)
ACCEPTED
REJECTED_BY_OR                  (terminal)
PENDING
REJECTED                        (terminal)
APPROVED
APPROVED-CANCELLATION-OR-REDUCTION
APPROVED-CANCELED              (terminal)
APPROVED-REDUCED               (terminal)
```

6.2 Transitions

Transition	From	To	Guard
submit_for_accepted	DRAFT	ACCEPTED	—
submit_for_reject_by_ord	DRAFT	REJECTED_BY_OR	—
submit_for_pending	ACCEPTED	PENDING	InvoiceGuard::canSubmitForPending (order_no set OR type == COMPTANT)
submit_for_approved	PENDING	APPROVED	—
submit_for_approved_cancellation	PENDING	APPROVED- CANCELLATION-OR- REDUCTION	—
submit_for_rejected	PENDING	REJECTED	—
submit_for_reduced	APPROVED / APPROVED- CANCELLATION...	APPROVED- REDUCED	—
submit_for_canceled	APPROVED / APPROVED- CANCELLATION...	APPROVED- CANCELED	—

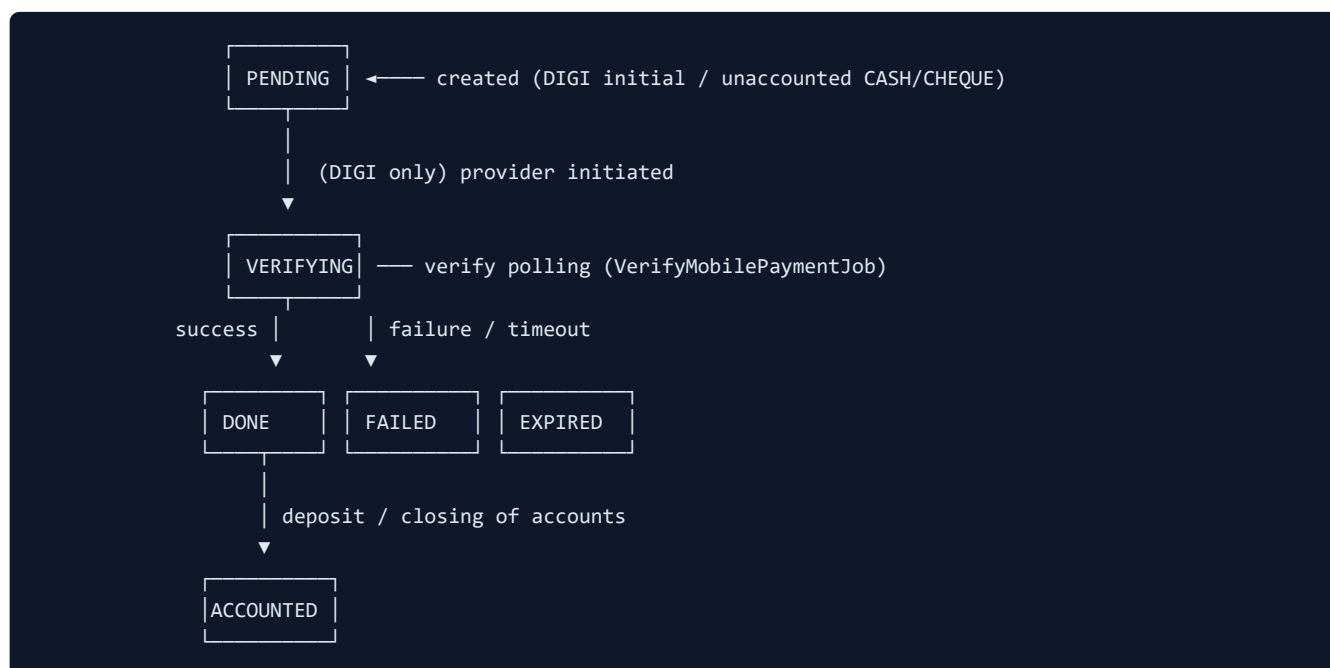
6.3 Transition diagram



Transitions are observed by `InvoiceWorkflowSubscriber` which fans out notifications and post-actions (release of taxables on rejection, agent notifications, etc.).

Payment can only be added when the invoice is in APPROVED (or APPROVED-CANCELLATION-OR-REDUCTION). This is checked by `Invoice::canGetPayment()` and by `PaymentPolicy`.

7. Payment lifecycle



`CANCELED` is used when a payment row is voided manually. Payments with `description = "Annulation"` or `"Réduction"` are not counted as effective payments by `getPrintData()` and `InvoiceCodeBalanceService`.

8. Cash & cheque payment procedure

Component: `app/Livewire/Payment/AddPaymentModal.php`.

1. Agent opens a taxpayer or invoice screen.
2. Triggers the **AddPaymentModal** (`payment_type` = CASH or CHEQUE).
3. Required fields:
 - `amount` (≤ remaining_amount on invoice).
 - `payment_type` ∈ { CASH , CHEQUE }.
 - `reference` (recommended for CHEQUE).
 - Optional `notes` .
4. On submit:
 1. The amount is **split per tax code** via `Invoice::getCode($invoice_id, $amount, $paymentData)` .
 2. One `Payment` row is created **per tax code**, all sharing the same `reference` and `created_at` .
 3. `Invoice.pay_status` is recomputed: OWING → PART PAID → PAID based on `getRestToPaid()` .
 4. `InvoiceCodeBalanceService::syncForInvoice($invoice)` is called to refresh per-code totals.
 5. If fully paid, `InvoicePaid` notification is dispatched.
5. Agent can immediately print the receipt PDF (see §11).

Status lifecycle for a CASH/CHEQUE row:

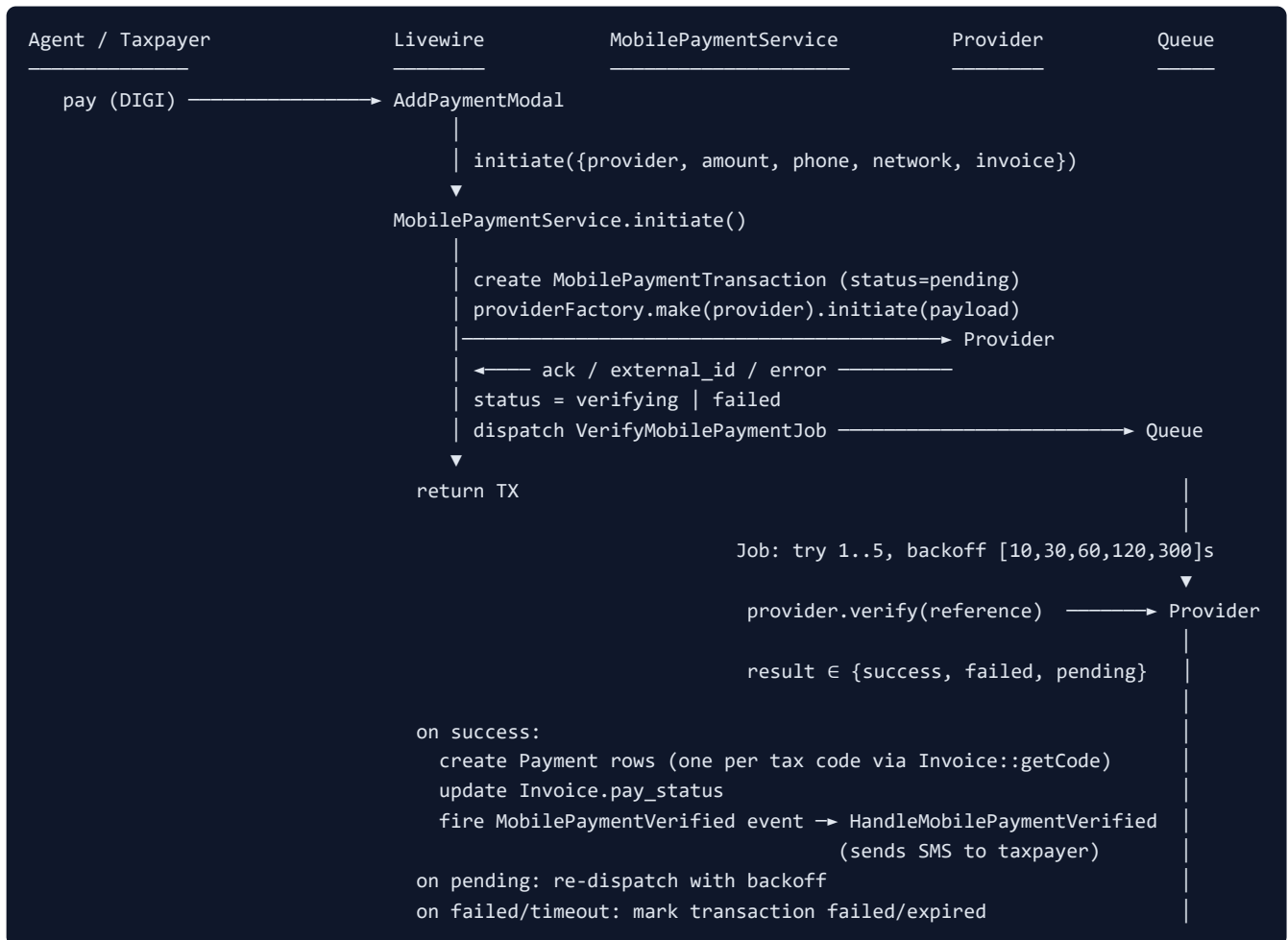
```
PENDING (created) → ACCOUNTED (added to a deposit/versement) → DONE (closed by accountant)
```

`CANCELED` is reachable manually if the policy allows it.

9. Mobile-money / digital payment procedure

End-to-end logic lives in `app/Services/MobilePayment/MobilePaymentService.php` and is **verification-based, never webhook-trusted**.

9.1 Sequence



9.2 Inputs validated by AddPaymentModal

- amount required, ≤ remaining.
- payment_type = DIGI.
- phone_number — 8 digits.
- network ∈ MOBILE_NETWORKS (TMONEY, FLOOZ).
- provider ∈ MOBILE_PROVIDERS (qosic, fedapay, paygate).

9.3 Provider abstraction

MobilePaymentProviderInterface exposes:

```

public function initiate(array $data): array; // returns ['ok'=>bool, 'external_id'=>?, 'response'=>...]
public function verify(string $reference): array; // returns ['status'=>'success|failed|pending', ...]

```

Concrete implementations:

- QosicProvider
- FedapayProvider
- PaygateProvider

The factory is MobilePaymentProviderFactory.

9.4 Retry & expiry policy

Defined in config/mobile-payment.php:

```
transaction_expiry_minutes = 1           # default
verification.max_attempts = 3
verification.backoff_seconds = [10, 30, 60, 120, 300]
```

`VerifyMobilePaymentJob` implements 5 attempts with the backoff above; expired transactions are swept by `ExpireStaleMobilePaymentsJob`.

9.5 Manual retry

`MobilePaymentActions` exposes a `retryVerification($id)` action (visible in the **mobile-payments** screen) for transactions stuck in `pending` / `verifying`.

10. Per-tax-code balance tracking

Service: `app/Services/Sync/InvoiceCodeBalanceService.php`.

For every invoice change (issuance, payment, cancellation, reduction), `syncForInvoice($invoice)`:

1. Aggregates `amount_billed` by `code` from `invoice_items` joined to `taxpayer_taxables` / `tax_labels`.
2. Aggregates `amount_paid` by `code` from `payments`, **excluding** rows whose `description` is `Annulation` or `Réduction`.
3. Upserts an `invoice_code_balances` row per (`invoice_id`, `code`) with `remaining_amount` and `status` (`OWING` / `PAID`).
4. Deletes stale balance rows for codes no longer present in the invoice.

This table is the source of truth for per-code reporting (ledgers, accountant deposits, recovery views).

11. Receipts and PDF generation

Service: `app/Services/PdfGeneratorService.php` (uses `Barryvdh\DomPDF\Facade\Pdf`).

Public API:

```
PdfGeneratorService::generateInvoicePdf(
    array $uuids,           // one or more invoice UUIDs
    string $templateName,   // resources/views/exports/{templateName}.blade.php
    ?int $action = null,    // 1 = notice, 2 = receipt
    bool $is_relance = false
): array // ['success'=>bool, 'pdf'=>stream, 'filename'=>string]
```

Behaviour:

- Eager-loads invoices via `Invoice::retrieveByUUIDs()`.
- Adds a QR code through `QrcodeGeneratorService` — used by collectors to reconcile in the field.
- On the first successful generation, sets `edition_state` = `'PRINT'` and `printed_at` = `now()`.
- Templates live in `resources/views/exports/` (e.g. `invoices.blade.php`).

Routes for printing:

- `GET /prints` — print queue UI.
- `GET /print-all-invoice` — bulk PDF download (`PrintController@downloadMultipleInvoicePdf`).

12. Synchronization with the remote collection app

A companion mobile app collects payments in the field and syncs back to SIG-RECETTE. See [docs/sync-entites-paiements.md](#) and [docs/sync-implementation.md](#).

12.1 Entities and direction

Entity	Direction	Purpose
Invoice	local → remote	Remote app shows unpaid notices
InvoiceItem	local → remote (read-only)	Detail lines
Taxpayer	local → remote (read-only)	Identification & contact
TaxpayerTaxable	local → remote (read-only)	Context of taxables
Payment	remote → local	Field-collected payments uploaded back

12.2 API endpoints — routes/api.php

```

GET /api/v1/sync/invoices      → Api/Sync/V1/SyncV1InvoicesController@index
GET /api/v1/sync/payments     → Api/Sync/V1/SyncV1PaymentsController@index (export local payments)
POST /api/v1/sync/payments    → Api/Sync/V1/SyncV1PaymentsController@store (import remote payments)

POST /api/v1/search/invoices  → SearchInvoiceController@search
POST /api/v1/search/taxpayers → SearchTaxpayerController@search
POST /api/v1/search/taxpayerstaxables → SearchTaxpayerTaxableController@search

```

12.3 Idempotency & filters

- Idempotent via `uuid` on `Invoice` and `Payment` (UUID v4 generated at creation).
- Incremental via `?updated_since=<ISO 8601>`.
- Active-year scoping via `Year::getActiveYear()`.
- Pagination via `?per_page=`.

12.4 Import service

PaymentImportService:

```

PaymentImportService::import(array $payments): [
    'created' => int,
    'updated' => int,
    'skipped' => int,
    'errors' => array,
]

```

For each item:

1. Resolve invoice by `invoice_uuid`.
2. `Payment::firstOrCreate(['uuid' => ...], $attributes)` and update if existing.
3. Recompute `Invoice.pay_status` and call `InvoiceCodeBalanceService`.

12.5 Background jobs

- `SyncImportPaymentsJob` — pulls remote payments page-by-page (timeout 120s, 3 tries).
- `SyncExportInvoicesJob` — pushes invoices to the remote.

Sync runs are tracked in `SyncRun` records used as a cursor.

13. Events, listeners, jobs and notifications

13.1 Events

Event	Purpose
<code>MobilePaymentVerified</code>	Fired when a <code>MobilePaymentTransaction</code> becomes <code>success</code>

13.2 Listeners

Listener	Reacts to	Action
<code>HandleMobilePaymentVerified</code>	<code>MobilePaymentVerified</code>	Sends SMS "Votre paiement de {amount} FCFA pour l'avis {invoice_no} a été confirmé. Ref: {reference}" (gated by <code>features.sms_notifications_feature</code>)
<code>InvoiceWorkflowSubscriber</code>	<code>workflow.invoice.</code> {enter,leave,transition,guard}	Notifies role-appropriate users (<code>InvoiceAccepted</code> , <code>InvoiceApproved</code> , <code>InvoiceRejected</code>), releases reserved <code>TaxpayerTaxable</code> rows on rejection, etc.

13.3 Jobs

Job	Purpose
<code>VerifyMobilePaymentJob</code>	Polls the provider — 5 tries, backoff [10, 30, 60, 120, 300] s
<code>ExpireStaleMobilePaymentsJob</code>	Marks transactions past <code>expires_at</code> as expired
<code>SyncImportPaymentsJob</code>	Pulls payments from remote
<code>SyncExportInvoicesJob</code>	Pushes invoices to remote

13.4 Notifications (database channel)

`InvoiceCreated` , `InvoiceAccepted` , `InvoiceApproved` , `InvoiceRejected` , `InvoicePaid` — all in [app/Notifications/](#).

14. Authorization and permissions

Permissions strings are in **French** and managed by Spatie.

14.1 InvoicePolicy — app/Policies/InvoicePolicy.php

Ability	Required permission
viewAny / view	open to any authenticated user
create	<i>peut émettre un avis au comptant OR peut émettre un avis sur titre</i>
update	<i>peut accepter un avis sur titre, peut prendre en charge..., peut ajouter date de livraison, peut ajouter numéro d'ordre</i>
delete	<i>peut rejeter un avis (and delegate variants)</i>
reduce	<i>peut réduire un avis (titre / comptant)</i>

14.2 PaymentPolicy — app/Policies/PaymentPolicy.php

Ability	Required permission
viewAny / view	any authenticated user
create	<i>peut ajouter un paiement</i>
update	<i>peut accepter un paiement OR peut comptabiliser un paiement</i>
delete	<i>peut supprimer un paiement</i>

14.3 InvoiceGuard — app/Guards/InvoiceGuard.php

`canSubmitForPending` blocks `ACCEPTED → PENDING` unless `order_no` is set or `type == COMPTANT`.

15. Routes reference

15.1 Web — routes/web.php

```

GET /invoices InvoiceController@index
GET /invoices/{id} InvoiceController@show
GET /mobile-payments MobilePaymentController@index

GET /accounts/collector-deposits CollectorDepositController@index
GET /accounts/collector-deposits/{id} CollectorDepositController@show
GET /accounts/accountant-deposits-title AccountantDepositController@index
GET /accounts/accountant-deposits-outright AccountantDepositOutrightController@index
GET /accounts/ledgers LedgerController@index

GET /recoveries RecoveryController@index
GET /prints PrintController@index
GET /print-all-invoice PrintController@downloadMultipleInvoicePdf

```

15.2 API — routes/api.php

```
GET /api/v1/sync/invoices      Api/Sync/V1/SyncV1InvoicesController@index
GET /api/v1/sync/payments     Api/Sync/V1/SyncV1PaymentsController@index
POST /api/v1/sync/payments    Api/Sync/V1/SyncV1PaymentsController@store

POST /api/v1/search/invoices  SearchInvoiceController@search
POST /api/v1/search/taxpayers SearchTaxpayerController@search
POST /api/v1/search/taxpayerstaxables SearchTaxpayerTaxableController@search
```

16. Configuration reference

16.1 config/mobile-payment.php

```
'default_provider' => env('MOBILE_PAYMENT_DEFAULT_PROVIDER', 'paygate'),
'transaction_expiry_minutes' => env('MOBILE_PAYMENT_EXPIRY_MINUTES', 1),

'verification' => [
    'max_attempts' => 3,
    'backoff_seconds' => [10, 30, 60, 120, 300],
],

'providers' => [
    'qosic' => ['base_url'=>env('QOSIC_BASE_URL'), 'username'=>env('QOSIC_USERNAME'), 'password'=>env('QOSIC_PASSWORD'), 'merchant_id'=>env('QOSIC_MERCHANT_ID')],
    'fedapay' => ['base_url'=>env('FEDAPAY_BASE_URL', 'https://sandbox-api.fedapay.com'), 'secret_key'=>env('FEDAPAY_SECRET_KEY'), 'public_key'=>env('FEDAPAY_PUBLIC_KEY')],
    'paygate' => ['base_url'=>env('PAYGATE_BASE_URL'), /* ... */ ],
],
```

16.2 config/workflow.php / config/workflow_registry.php

Declare the `invoice` state machine (states & transitions listed in §6).

16.3 config/sync.php

Holds endpoints, tokens and pagination for the remote collection app.

16.4 Required `.env` keys (payment-related)

```
MOBILE_PAYMENT_DEFAULT_PROVIDER=paygate
MOBILE_PAYMENT_EXPIRY_MINUTES=1

QOSIC_BASE_URL=
QOSIC_USERNAME=
QOSIC_PASSWORD=
QOSIC_MERCHANT_ID=

FEDAPAY_BASE_URL=https://sandbox-api.fedapay.com
FEDAPAY_SECRET_KEY=
FEDAPAY_PUBLIC_KEY=

PAYGATE_BASE_URL=
PAYGATE_*=...

QUEUE_CONNECTION=database # required for VerifyMobilePaymentJob
```

17. Database migrations reference

17.1 `invoices` (anchor migration)

[2024_02_03_000548_create_invoices_table.php](#)

```
id          BIGINT PK AUTOINC (starts at 100001)
invoice_no  VARCHAR
order_no    VARCHAR
status      VARCHAR DEFAULT 'DRAFT'
pay_status  VARCHAR DEFAULT 'OWING'
delivery     VARCHAR DEFAULT 'NOT DELIVERED'
delivery_date DATE
from_date   DATE
to_date     DATE
qty         DOUBLE
amount      DOUBLE DEFAULT 0
taxpayer_id FK → taxpayers.id
created_at, updated_at
```

Progressive migrations add `validity`, `reduce_amount`, `delivery_to`, `uuid`, `type`, `is_on_recovery_print`, `edition_state`, `notes (json)`, `reason_for_reject`, `printed_at`.

17.2 `payments`

[2024_02_26_005552_create_payments_table.php](#) — base columns, then incremental migrations add `remaining_amount`, `user_id`, `uuid`, `code`, `invoice_type`, `notes`, deposit reference; finally:

- [2026_03_28_000001_add_mobile_payment_fields_to_payments_table.php](#) — `provider`, `phone_number`, `external_id`, `verification_attempts`, `last_checked_at`, `expires_at`.
- [2026_04_08_000001_add_network_column_to_payments_and_mobile_payment_transactions_tables.php](#) — `network`.

17.3 `mobile_payment_transactions`

[2026_03_28_000002_create_mobile_payment_transactions_table.php](#)

```
id, reference UNIQUE,
invoice_id FK, taxpayer_id FK NULL, user_id FK NULL,
amount DECIMAL(15,2), phone_number, provider, network,
external_id, status DEFAULT 'pending',
verification_attempts SMALLINT DEFAULT 0,
last_checked_at, verified_at, expires_at,
provider_response JSON, meta JSON,
INDEX (invoice_id, status), INDEX (status, expires_at), INDEX (provider)
```

17.4 `invoice_code_balances`

[2026_03_01_000000_create_invoice_code_balances_table.php](#)

```

invoice_id FK, taxpayer_id FK,
code, year,
amount_billed, amount_paid, remaining_amount,
status DEFAULT 'OWING', last_payment_at,
UNIQUE (invoice_id, code)
INDEX (taxpayer_id, code, year)
INDEX (taxpayer_id, year)

```

18. Step-by-step procedures

18.1 Issue a new invoice and collect a cash payment (COMPTANT)

1. **Open a taxpayer file** → click *Nouvel avis*.
2. Fill the `AddInvoiceModal` (taxpayer, taxables, period). Choose `type = COMPTANT`.
3. Submit → invoice created in `DRAFT`.
4. Apply transitions:
 - `submit_for_accepted` (`DRAFT` → `ACCEPTED`).
 - `submit_for_pending` (`ACCEPTED` → `PENDING`) — **allowed because** `type = COMPTANT` even without `order_no`.
 - `submit_for_approved` (`PENDING` → `APPROVED`).
5. Open `AddPaymentModal`, choose `CASH`, type the amount, save.
6. The system creates one `Payment` per tax code, recomputes `pay_status` and refreshes `invoice_code_balances`.
7. Print the receipt (§11).

18.2 Issue a TITRE invoice (must have an order number)

1. Same as above with `type = TITRE`.
2. After `ACCEPTED`, the agent **must** set `order_no` (permission *peut ajouter numéro d'ordre*).
3. Then `submit_for_pending` succeeds; otherwise the `InvoiceGuard::canSubmitForPending` blocks the transition.
4. Continue with approval and payment.

18.3 Collect a digital payment (DIGI)

1. Open `AddPaymentModal` on an `APPROVED` invoice.
2. Pick `DIGI`, choose provider (`paygate` default), network (`TMONEY` / `FLOOZ`), enter the **8-digit** phone.
3. Save → `MobilePaymentService::initiate()` creates a `MobilePaymentTransaction` (pending), calls the provider, and queues a `VerifyMobilePaymentJob`.
4. The taxpayer validates the USSD/app prompt on their phone.
5. The job polls the provider with backoff. On `success`, `Payment` rows are created, `pay_status` updated, `MobilePaymentVerified` event fires, the taxpayer receives an SMS confirmation.
6. If polling exhausts attempts or `expires_at` passes, the transaction is marked `failed` / `expired`. The agent can use `Retry` from the **Mobile payments** page.

18.4 Cancel or reduce an approved invoice

1. From `APPROVED`, transition `submit_for_approved_cancellation` → `APPROVED-CANCELLATION-OR-REDUCTION`.
2. Then either:
 - `submit_for_canceled` → `APPROVED-CANCELED`, **and** create a corrective `Payment` with `description = "Annulation"`.
 - `submit_for_reduced` → `APPROVED-REDUCED`, **and** create a corrective `Payment` with `description = "Réduction"`, plus update `Invoice.reduce_amount`.
3. These `Annulation` / `Réduction` rows are excluded from `getPrintData()` and from the per-code balance computation.

18.5 Close a collector's deposit

1. Open *Comptes* → *Versements régisseur* (`/accounts/collector-deposits`).

2. Pick the date range, the system lists `Payment` rows where `r_user_id = current collector` and `status = PENDING`.
3. Validate the deposit: `status` becomes `ACCOUNTED`, the `deposit` column is filled.
4. Accountant deposits are then closed under *Versements comptable* (titre / comptant) which moves them to `DONE`.

18.6 Synchronize with the remote collection app

1. Schedule `SyncExportInvoicesJob` so the remote always sees current `APPROVED` notices.
2. Schedule `SyncImportPaymentsJob` every X minutes — it calls `GET /api/v1/sync/payments?updated_since=...&per_page=...`, then `PaymentImportService::import()`.
3. Cursor (`updated_since`) is persisted via `SyncRun`.

19. Troubleshooting

Symptom	Likely cause	Where to look
Cannot move invoice from <code>ACCEPTED</code> to <code>PENDING</code>	<code>order_no</code> missing on a <code>TITRE</code>	<code>InvoiceGuard::canSubmitForPending</code>
<code>AddPaymentModal</code> rejects amount	<code>Amount > Invoice::get_remains_to_be_paid()</code>	Validation rules in <code>AddPaymentModal</code>
DIGI stuck in <code>verifying</code>	Job queue not running, or provider timeout	Run <code>php artisan queue:work</code> ; inspect <code>mobile_payment_transactions.last_checked_at</code> , <code>provider_response</code> ; click <i>Retry</i>
DIGI ends expired	<code>transaction_expiry_minutes</code> too low or taxpayer didn't validate	<code>config/mobile-payment.php</code> , <code>ExpireStaleMobilePaymentsJob</code>
Per-code totals look stale	<code>InvoiceCodeBalanceService::syncForInvoice()</code> not called	Trigger by saving any <code>Payment</code> on the invoice; or re-sync via <i>tinker</i>
Sync import says skipped	Same <code>payment.uuid</code> already imported (idempotency)	Expected behaviour
PDF receipt shows Annulation lines	These rows are normally hidden; check <code>description</code> filter in <code>Payment::getPrintData()</code>	<code>app/Traits/PaymentTrait.php</code>
Notification never sent	<code>features.sms_notifications_feature</code> disabled or SMS provider creds missing	<code>config/features.php</code> , <code>SmsService</code>

Appendix — Quick reference cheat-sheet

```

Invoice states : DRAFT → ACCEPTED → PENDING → APPROVED → (CANCELED|REDUCED)
Pay status    : OWING → PART PAID → PAID
Payment types : CASH | CHEQUE | DIGI
DIGI providers : qosic | fedapay | paygate
DIGI networks  : TMONEY | FLOOZ
DIGI tx status : pending → verifying → success | failed | expired
Payment status : PENDING → ACCOUNTED → DONE (CANCELED, plus DIGI-only VERIFYING/FAILED/EXPIRED)
Idempotency key : uuid (Invoice & Payment)
Per-code totals : invoice_code_balances (unique on invoice_id+code)

```

Document generated for the SIG-RECETTE project.